Problema Insula

Propusă de: stud. Luișcan Alexandru-Mihai, Universitatea "Babeș-Bolyai", Cluj-Napoca

Subtaskurile 1 și 2. (22 de puncte)

Pentru a determina camerele utile este suficient să implementăm în cod exact definiția din enunț, adică să verificăm pentru fiecare cameră dacă este cumva acoperită de o altă cameră.

Specific, se vor itera șirul cu toate camerele (și să spunem că ne aflăm pe poziția $1 \leq i \leq n$). Pentru a determina dacă camera este utilă vom analiza toate camerele (mai puțin i), și vom verifica dacă există o cameră j ($1 \leq j \leq n$) cu proprietatea $s_j \leq s_i$ și $e_i \leq e_j$. În funcție de aceste rezultat ne putem da seama dacă camera i este utilă.

Odată identificate camerel utile, acestea vor fi inserate într-un alt vector, urmând ca apoi să fie sortate conform cerinței pentru a le afișa în ordine.

Complexitatea calculului camerelor utile este $O(n^2)$ iar sortarea se poate realiza în $O(n \cdot log(n))$ sau în $O(n^2)$. Ambele cazuri conduc la o complexitate finală de $O(n^2)$

Subtaskul 3. (18 puncte)

O metodă mai eficientă de a determina camerele utile se poate realiza prin pre-sortarea camerelor. Acestea vor fi sortate, în primul rând în funcție de s_i - *crescător*; iar dacă există 2 camere cu $s_i = s_j$, atunci vor fi în funcție de e_i - *descrecător*.

Motivația din spatele acestei sortări este următoarea: Odată ce am găsit o cameră (i) și am determinat că este utilă, atunci toate camere care încep la $s_j \geq s_i$, vor putea fi utile doar dacă $e_j > e_i$ (altfel intervalul acestor camera ar fi inclus în intervalul determinat de camera i). Modul de construcție al algoritmului este următorul: se pornește cu un vector gol de camera utile în care se adaugă prima cameră (cea cu s_i minim și, dacă există mai multe, cea cu e_i maxim). Apoi, se iterează camerele în ordinea explicată mai sus, iar dacă întâlnim o cameră j atunci dacă:

1. $e_j \leq e_i$ - camera j nu este utilă
2. $e_j > e_i$ - camera j este utilă

Aici e_i reprezintă sfârșitul ultimei camere din vectorul de camere utile. Dacă camera j este utilă atunci se va adăuga în vectorul rezultat. Apoi algoritmul continuă pentru toate celelalte camere.

La final, tot ce mai trebuie făcut este afișarea vectorului.

Complexitatea algoritmului este $O(n \cdot log(n))$ pentru sortare și $O(n)$ pentru construcția rezultatului. În final $O(n \cdot log(n))$.

Observație: Se poate observa că există o mică discrepanță între modul în care se cere să fie afișate datele și modul în care camerele sunt sortate de algoritm. Cerința cere ca rezultatul să fie afișat sortat după s_i - *crescător* și apoi după e_i - *crescător*; pe când algoritmul descris realizează sortarea după s_i - *crescător* și apoi după e_i - *descrescător*. Ce trebuie dedus de aici este că în vectorul rezultat care trebuie afișat, nu vor exista două camere distincte care să aibă $s_i = s_j$. Acest lucru se explică datorită faptului că, din definiție, o cameră este utilă dacă nu este acoperită de o alta. Atunci, dacă există două camere care încep la aceeași poziție ($s_i = s_j$) atunci putem spune sigur că una dintre ele este reduntantă, și cealaltă utilă.

Subtaskurile 4, 5, 6. (30 de puncte)

Pentru această cerință se cere determinarea intervalului specific. Vom porni cu asumpția că avem deja calulat vectorul de camere utile (pe baza descrierii de mai înainte), sortat după poziția de start a camerei (vezi observația de mai sus pentru justificarea că nu vor exista două camere cu aceeași poziție de start, ambele utile).

Pentru această cerință, algoritmul de construcție a soluție este următorul:

1. Se identifică prima cameră care începe după poziția X. Acest lucru se poate realiza printr-o căutare secvențială.
2. Conform cerinței, trebuie găsită următoarea cameră care începe la $s_j \leq e_i$ si care are e_j maxim. Din nou, acest pas se poate realiza printr-o căutare liniară, pornind de la prima cameră care începe după ultima cameră salvată.
3. Camera astfel găsită se salvează și se repetă pasul 2 de $K - 2$ ori pentru a determina toate cele K camere care formează rezultatul.
4. Dacă la un moment dat, în pasul 2, nu se găsește o cameră care să respecte condiția (s-au terminat toate camerele sau există un gol neacoperit de nicio cameră) atunci rezultatul algoritmului este 0 (conform cerinței).

La final, se afișează diferența dintre începutul intervalului acoperit de prima cameră și sfârșitul intervalului acoperit de camera K (sau 0 dacă nu au fost găsite cele K camere).

Deoarece problema nu necesită afișarea camerelor care formează intervalul, iar algoritmul lucrează doar cu prima, respectiv, ultima cameră descoperită, în cod nu va fi necesar construcția vectorului auxiliar, fiind suficientă salvarea datelor referitoare la doar cele 2 camere.

Complexitatea finală a algoritmului este $O(n \cdot log(n))$ sau $O(n^2)$ pentru determinarea camerelor utile și $O(n \cdot k)$ pentru construcția rezultatului. În total $O(n \cdot (log(n) + K))$ sau $O(n \cdot (n + K))$.

Ca un mic detaliu, deoarece unul dintre subtaskuri garantează că camerele inițiale sunt toate utile, atunci se pot obține 12 puncte fără implementarea algoritmului corespunzător cerinței 1.

Subtaskurile 7 și 8. (30 de puncte)

Pentru a obține punctajul maxim, este suficientă o mică optimizare asupra algoritmului precedent. Singura schimbare necesară este înlocuirea căutării secvențiale din pasul 2 al algoritmului cu căutarea binară.

Acest lucru se poate face pe baza următorului raționament. Vectorul de camere utile este sortat în funcție de poziția de start (s_i) iar elementul căutat este acea cameră j care respectă $s_j \leq e_i$ și posedă e_j maxim. Putem realiza căutare binară să identificăm toate camerele care respectă condiția de existență $s_j \leq e_i$ (practic, putem calcula ultimul indice în vectorul de camere utile în care găsim acele camere j care respectă condiția); dar acest lucru nu este suficient pentru a o găsi pe cea cu e_j maxim. Acest lucru se poate face doar în urma observației că, deoarece camerele sunt utile, camera cu e_j maxim va fi și cea cu s_j maxim (dintre camerele care respectă condiția $s_j \leq e_i$, nu dintre toate camerele utile).

Demonstrația se poate realiza prin reducere la absurd: dacă există două camere j_1 și j_2 utile care respectă condițiile $s_j \leq e_i$ și $s_{j1} \leq s_{j2}$, atunci dacă $e_{j1} > e_{j2}$ camera j_2 este complet acoperită de camera j_1 ($s_{j1} \leq s_{j2}$ și $e_{j2} < e_{j1}$) de unde rezultă că camera j_2 nu este utilă, ceea ce contrazice ipoteza. De aici deducem că camera j cu e_j maxim căutată este camera j cu s_j maxim, și, având în vedere că vectorul nostru

este sortat în funcție de s_i rezultă că se poate aplica algoritmul de căutare binară pentru a determina rezultatul pasului 2. Tot ce mai trebuie este verificarea condiției că în urma căutării binare camera găsită este validă (altfel rezultatul algoritmului este 0).

Complexitatea finală a algoritmului este $O(n \cdot log(n))$ pentru calculul camerelor utile, $O(n)$ pentru determinarea primei camere din interval (acest pas se poate realiza și prin căutare binară în $O(log(n)$ dar nu este necesar), și $O(K \cdot log(n))$ pentru calculul intervalului. În final $O((n + K) \cdot log(n))$.

Deoarece unul din subtaskuri precizează că în vectorul de camere inițiale toate sunt utile, atunci se pot obține 10 puncte prin aplicarea algoritmului corespunzător intervalului, fără rezolvarea primei cerințe.

Contributori: *stud. Andrei Horodniceanu, stud. Alexandru Lazăr, Universitatea "Babeș-Bolyai", Cluj-Napoca*